

SAÉ S2.04 — Exploitation d'une base de données

31/05/2026

Analyse des stations-service et des prix des carburants à partir de données ouvertes

Livrable 3 — Base de données et requêtes SQL

Data Guardians

Abasse KADERBHAI

[REDACTED]
[REDACTED]

Groupe 3 CERES
BUT INFORMATIQUE

Table des matières

1. Introduction	2
2. Objectif du livrable	2
3. Contenu de l'archive	2
4. Répartition du travail	2
5. Création de la base de données.....	3
6. Importation des données	3
7. Historisation des prix	3
8. Requêtes SQL d'analyse	4
9. Premiers résultats obtenus	4
10. Difficultés rencontrées et solutions	4
11. Conclusion	5

1. Introduction

Ce troisième livrable présente la mise en place de la base de données du projet.

La base doit permettre de stocker les informations sur les stations-service, les carburants, les prix, les ruptures, les horaires d'ouverture et les services proposés.

Pour faciliter l'exécution du projet, les commandes à lancer sont regroupées dans le fichier *README.md*, placé à la racine de l'archive. Ce fichier indique l'ordre à suivre pour créer la base, importer les données et exécuter les requêtes d'analyse.

2. Objectif du livrable

Après modélisation réalisée dans le livrable 2, l'objectif est de passer de la conception à une base réellement exploitable.

Ce travail doit permettre de :

- Créer les tables de la base
- Ajouter les contraintes d'intégrité
- Importer les données
- Conserver l'historique des prix
- Produire les premières requêtes d'analyse

3. Contenu de l'archive

L'archive ZIP est organisée en plusieurs dossiers.

DOSSIER	CONTENU
doc/	Document PDF du livrable
SQL/	Script SQL de création, d'importation et d'analyse
Python/	Script Python utilisé pour nettoyer les horaires
data/	Fichiers de données utilisés
README.md	Explication du contenu de l'archive et procédure d'exécution

Cette organisation permet de retrouver facilement les différents fichiers du projet.

4. Répartition du travail

Membre	Tâches
Abasse	Création de la base de données, tables SQL, clés et contraintes
Amine	Importation, traitement des données et nettoyage des horaires
Seevaghan	Requêtes SQL d'analyse et premiers résultats
Tous les membres	Test des scripts, vérification de la base et relecture finale

5. Création de la base de données

La base de données reprend principalement le modèle défini dans le livrable 2.

Les tables créées sont :

- STATION
- CARBURANT
- RELEVES_PRIX
- RUPTURE
- HORAIRE
- SERVICE
- STATION_SERVICE

Après l'analyse des données réellement disponibles, quelques ajustements ont été nécessaires. L'attribut 'nom' prévu dans la table *STATION* n'a pas été conservé, car le fichier exploité ne contenait pas de nom de station directement exploitable. Les stations sont donc identifiées par leur identifiant, leur adresse, leur ville, leur code postal et leurs coordonnées géographiques.

Les coordonnées géographiques présentes dans le fichier source sont stockées sous forme d'entiers. Lors de l'importation, elles sont divisées par 100 000 afin d'obtenir des valeurs de latitude et longitude exploitables pour les futures visualisations cartographiques dans Grafana.

De plus, la date de fin de rupture prévue dans le livrable 2 n'était pas présente dans les données. Elle a donc été remplacée par l'attribut 'type', qui indique la nature de la rupture.

Voici le nouveau schéma relationnel de la table *STATION* et *RUPTURE* :

- STATION (id_station, adresse, ville, code_postal, longitude, latitude)
- RUPTURE (id_rupture, #id_station, #id_carburant, date_debut, type)

6. Importation des données

Les données principales proviennent du fichier open data des prix des carburants (<https://www.data.gouv.fr/datasets/prix-des-carburants-en-france-flux-instantane-v2-amelioree>).

Avant d'être insérées dans les tables finales, certaines données sont placées dans des tables intermédiaires avec le type 'TEXT' pour faciliter la lecture. Cela permet de mieux organiser l'importation et de séparer ensuite les informations dans les bonnes tables.

Les horaires sont présents dans le fichier source au format JSON. Pour les rendre plus simples à importer et pouvoir les exploiter, un script python a été utilisé. Ce script transforme les horaires en un fichier CSV plus clair et plus détaillé, avec une ligne par station, jour et plage horaire.

Pour l'importation des prix et ruptures, nous avons utilisé *CROSS JOIN LATERAL* afin de transformer les colonnes liées aux différents carburants en lignes exploitables dans les tables finales.

7. Historisation des prix

L'historisation des prix est gérée grâce à la table *RELEVES_PRIX*.

Chaque prix est enregistré avec :

- La station concernée
- Le carburant concerné

- Le prix
- La date de mise à jour

Lorsqu'un nouveau prix est importé, l'ancien n'est pas supprimé. Une nouvelle ligne peut être ajoutée avec une nouvelle date.

Ce fonctionnement permet de suivre l'évolution des prix dans le temps, ce qui est demandé dans le sujet. Le projet doit en effet permettre de conserver les collectes afin de réaliser une analyse temporelle.

8. Requêtes SQL d'analyse

Plusieurs requêtes SQL ont été préparées pour exploiter les données.

Elles permettent notamment de :

- Afficher les stations avec les carburants disponibles
- Comparer les prix entre plusieurs stations
- Trouver les stations les moins chères pour un carburant donné
- Analyser les ruptures de carburant
- Exploiter les horaires d'ouverture
- Calculer des indicateurs simples comme la moyenne, le minimum et le maximum des prix
- Compter le nombre de stations

Ces requêtes correspondent aux analyses attendues dans le sujet : analyse des stations, disponibilités, horaires, comparaisons et indicateurs.

Les résultats détaillés sont obtenus directement lors de l'exécution du fichier '04_requetes_analyses.sql'.

9. Premiers résultats obtenus

Les premières requêtes permettent d'obtenir des informations utiles sur les stations-service.

Elles permettent par exemple de comparer les prix d'un même carburant entre plusieurs stations et d'identifier les stations les moins chères.

Les requêtes sur les ruptures permettent de repérer les stations où certains carburants sont indisponibles.

Les indicateurs comme la moyenne, le minimum et le maximum donnent une première vision générale des prix selon les carburants.

Ces premiers résultats serviront ensuite à construire les tableaux de bord du livrable 4.

10. Difficultés rencontrées et solutions

Une difficulté rencontrée concerne les horaires d'ouverture. Dans le fichier source, les horaires ne sont pas directement exploitables sous forme de table simple.

Pour résoudre ce problème, un script Python a été utilisé afin d'extraire les horaires et de les transformer en fichier CSV.

Une autre difficulté concerne l'organisation des données, car le fichier source contient plusieurs informations mélangées. L'utilisation de tables intermédiaires permet de faciliter l'importation avant de répartir les données dans les tables finales.

Une autre amélioration a été apportée au script d'importation des relevés de prix et des ruptures. Au départ, l'insertion des données nécessitait plusieurs requêtes répétées avec *UNION ALL*, car chaque carburant était stocké dans une colonne différente du fichier source. Pour rendre le code plus compact et plus lisible, nous avons recherché une solution SQL plus adaptée. Nous avons donc utilisé *CROSS JOIN LATERAL* avec *VALUES*, ce qui permet de transformer plusieurs colonnes de carburants en plusieurs lignes à insérer dans les tables *RELEVES_PRIX* et *RUPTURE*. Cette solution évite les répétitions inutiles et rend le script plus facile à maintenir.

11. Conclusion

Ce livrable permet de passer de la modélisation à une base de données fonctionnelle.

Les scripts SQL permettent de créer les tables, d'ajouter les clés et contraintes, puis d'importer les données. Les premières requêtes montrent que la base peut être exploitée pour analyser les stations-service, les carburants, les prix, les ruptures et les horaires.

Ce travail servira de base pour le livrable 4, qui portera sur les visualisations et les tableaux de bord de Grafana.